



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 04

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 12/MARZO /2026

CALIFICACIÓN: _____

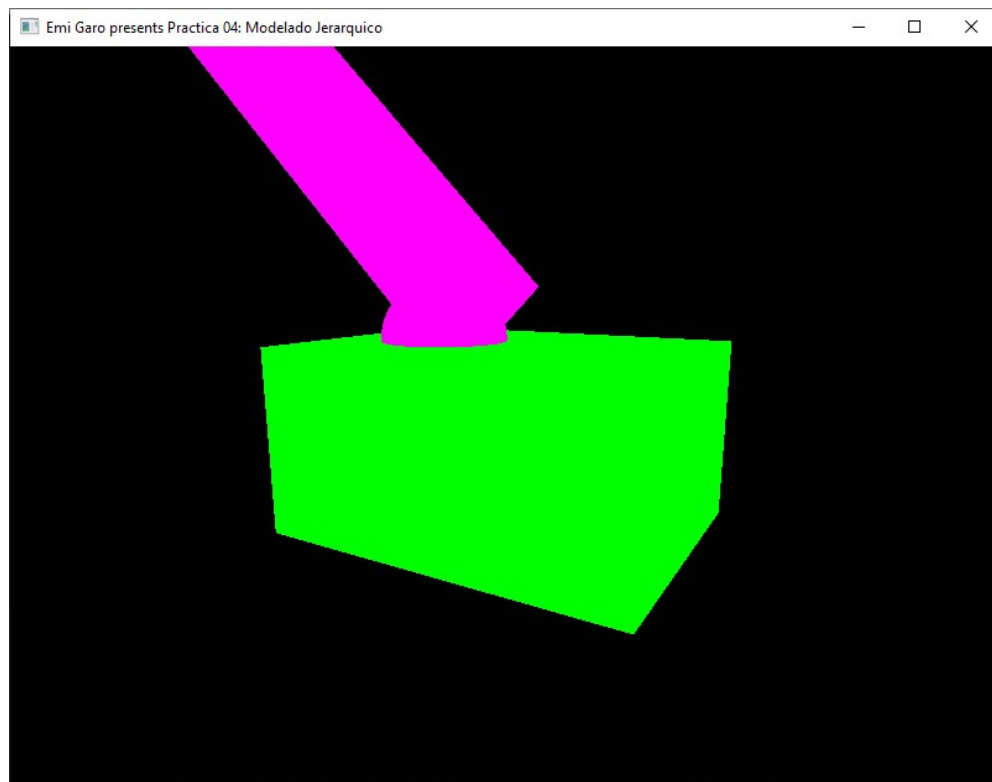
1.- Terminar la Grúa

Cuerpo

Para el cuerpo simplemente utilizaremos un prisma rectangular, del que ademas de ser el cuerpo funcionara como el nodo padre del que todas las demas primitivas seran jerarquia.

```
// CABINA - Nodo padre -----
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 3.0f, 0.0f));
x = mainWindow.getarticulacionX();
y = mainWindow.getarticulacionY();
model = glm::translate(model, glm::vec3(x, 0.0f, y));
modelaux = model;
modelaux2 = model;
model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(0.0f, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh();
model = modelaux;
```

Podemos destacar el uso de coordenadas x, y con las que podremos mover la grua entera a partir del nodo padre con las flechas, simulando movimiento. Ademas de esto tenemos dos matrices auxiliares, una para la parte superior y otra para la parte inferior.



Esto configurando las teclas en el archivo window.c y definiendolas en el header.

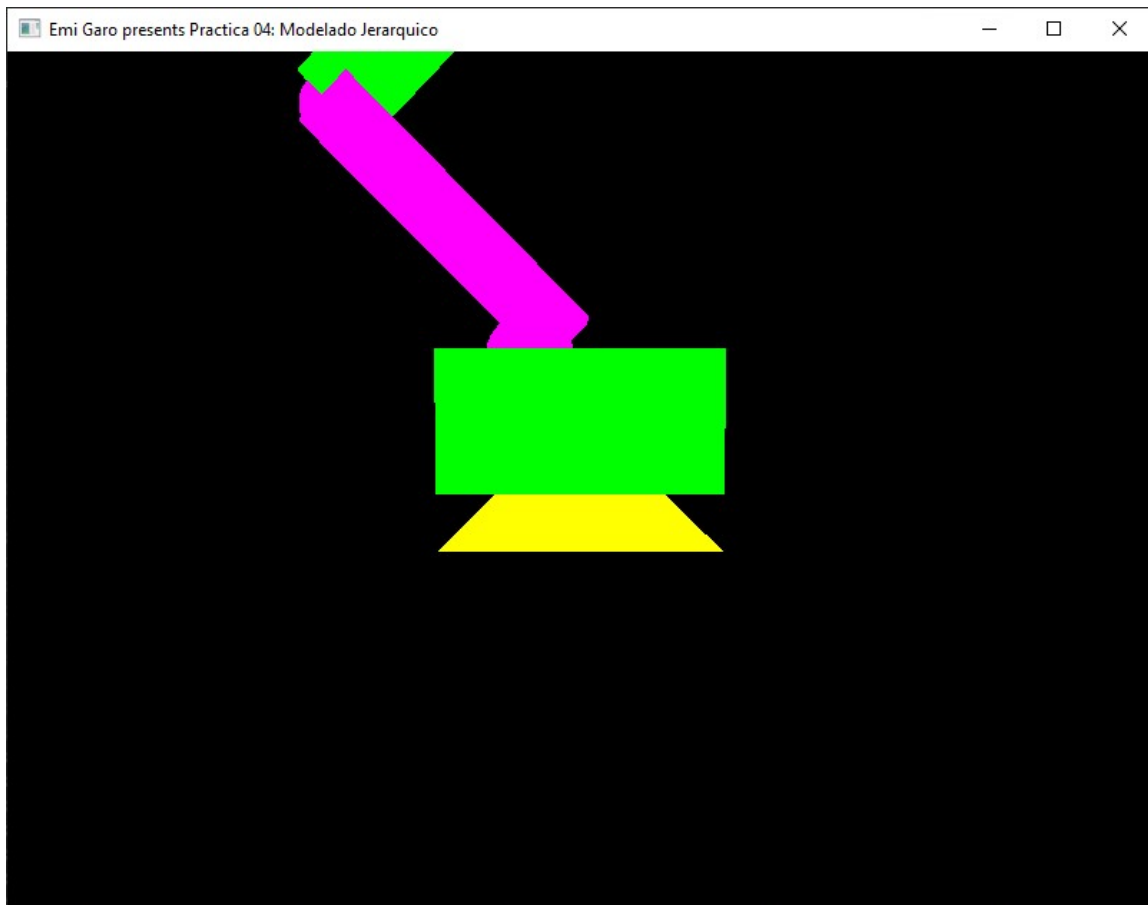
```
if (key == GLFW_KEY_RIGHT){ theWindow->articulacionY -= 0.5; }
if (key == GLFW_KEY_LEFT) { theWindow->articulacionY += 0.5; }
if (key == GLFW_KEY_UP) { theWindow->articulacionX -= 0.5; }
if (key == GLFW_KEY_DOWN) { theWindow->articulacionX += 0.5; }
```

Base

Enseguida de esto crearemos la base utilizando una piramide rectangular de color amarillo.

```
// Base
model = glm::translate(model, glm::vec3(0.0f, -0.8f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(4.0f, 2.0f, 3.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
                    glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1, 1.0f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[4]->RenderMesh();
```

A partir de aquí estaremos trabajando con jerarquias por lo que las matrices comenzaran a utilizar la matriz de su respectivo nodo padre. Una vez hecho esto, la posicionamos justo debajo del cuerpo y la mandamos a renderizar.

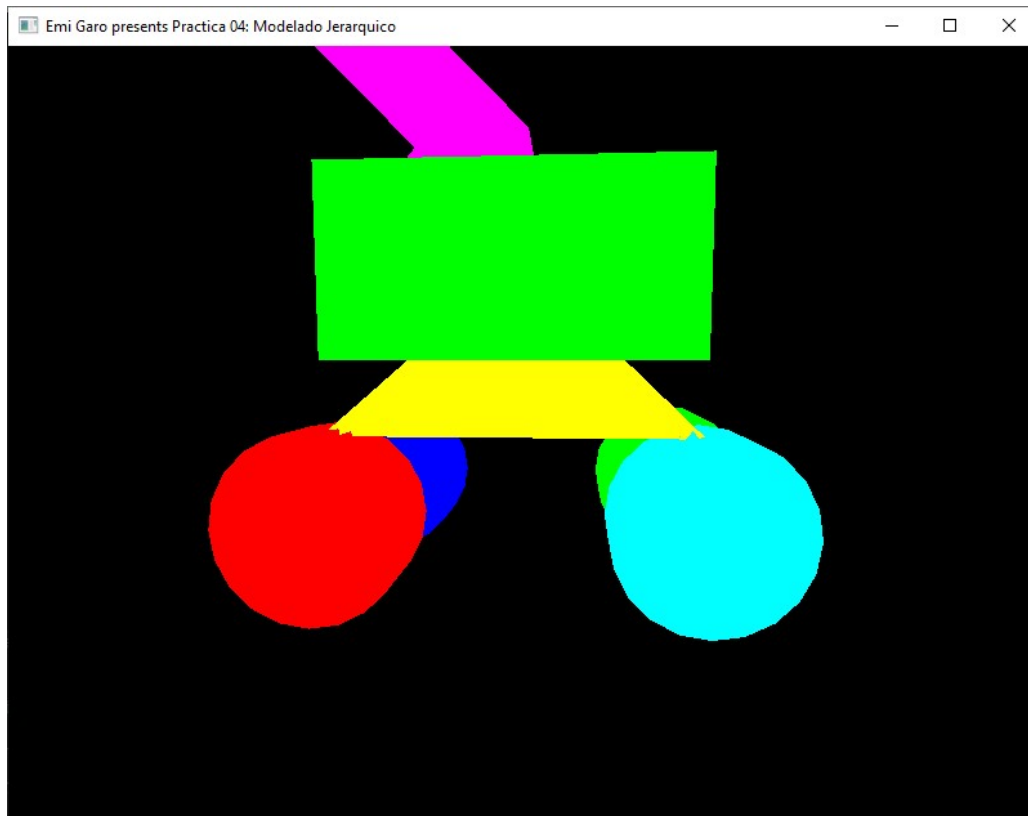


Llantas

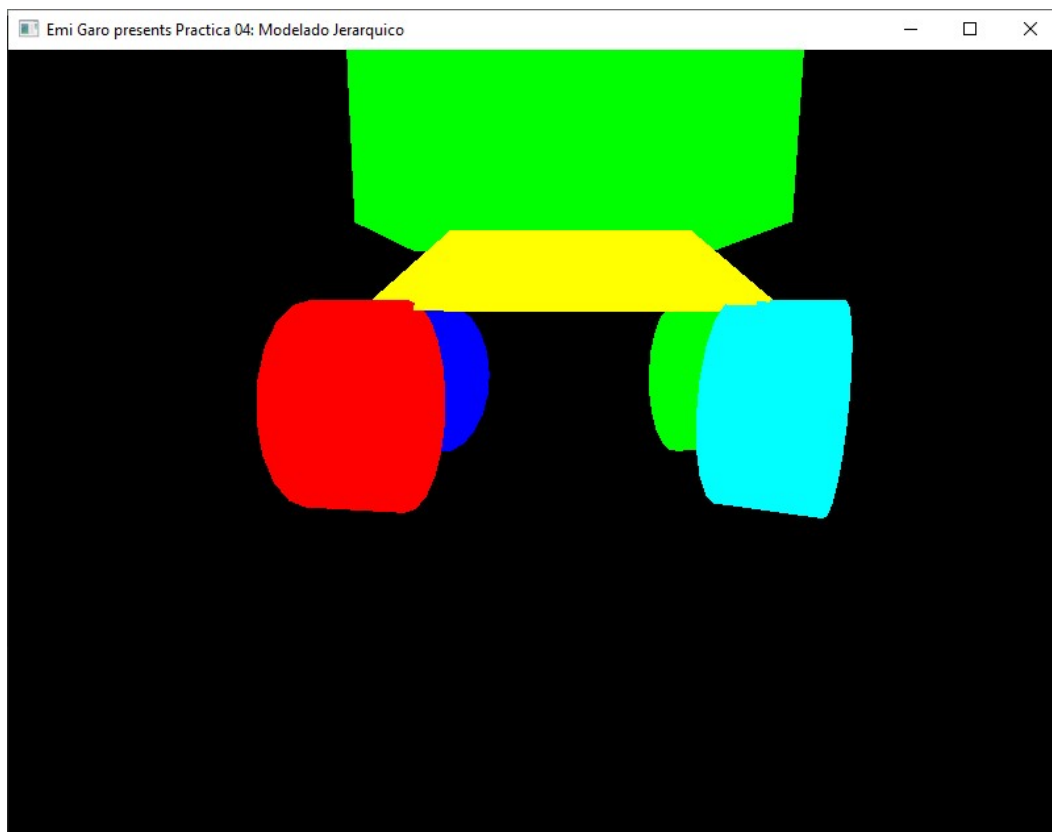
Comenzaremos a modelar las llantas. Estas heredan de la matriz de la base y posicionamos cualquiera de las cuatro. Una vez tenemos una repetimos el código 3 veces más invirtiendo valores en x, y para que sean simétricas.

```
//Llantas4
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -1.9f, -1.5f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()),
                    glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0, 0.f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, -1.9f, -1.5f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()),
                    glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0, 1.f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(-2.0f, -1.9f, 1.5f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()),
                    glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(1, 0.f, 0.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
model = modelaux;
model = glm::translate(model, glm::vec3(2.0f, -1.9f, 1.5f));
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()),
                    glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
color = glm::vec3(0, 1.f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
```

Además de modelar las llantas notaremos las líneas con la función `glm::rotate()`, donde el argumento de radianes es un valor variable que podremos controlar con las teclas que especifiquemos en `window.c`. Estas como en el ejercicio de la práctica solo indican que basado en la tecla especificada, roten de acuerdo al `vec3` en la misma función, en este caso (0.0, 0.0, 1.0).

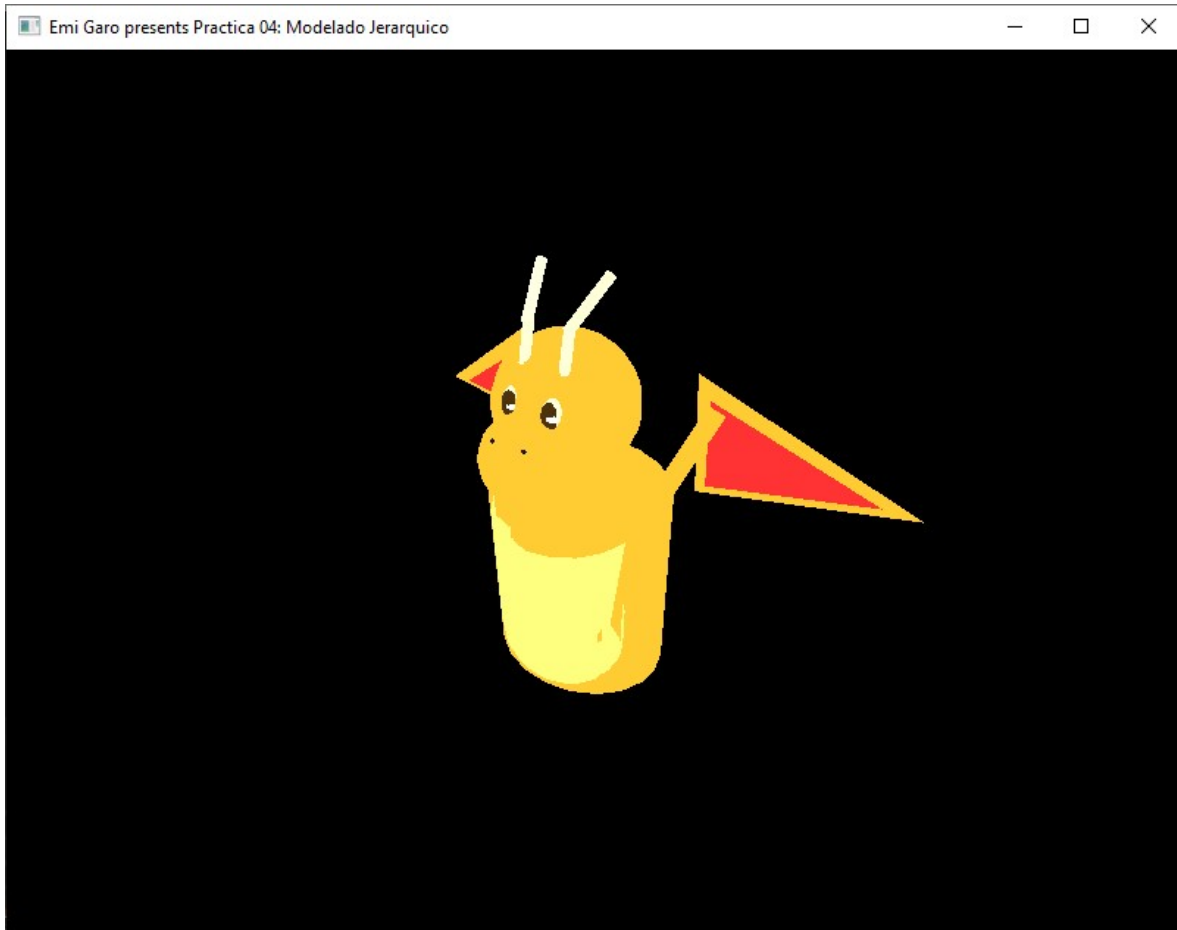


Ahora si presionamos las teclas con las que giramos las ruedas (K, L, N y M).



2.- Crear un animal robot 3d

Primero modelamos con las primitivas trabajadas en laboratorio las partes estaticas del modelo, de esta manera no introducimos complejidad al principio y podemos enfocarnos en la forma del animal. En mi caso escogi un dragonite de pokemon y con las partes estaticas (cuerpo, cabeza, alas, ojos, antena, etc) se ve asi.



El codigo con es diferente del que hemos estado trabajando anteriormente y se conforma en su mayoria de esferas y cilindros. Se veria asi por ejemplo.

```
// Head
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 3.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
```

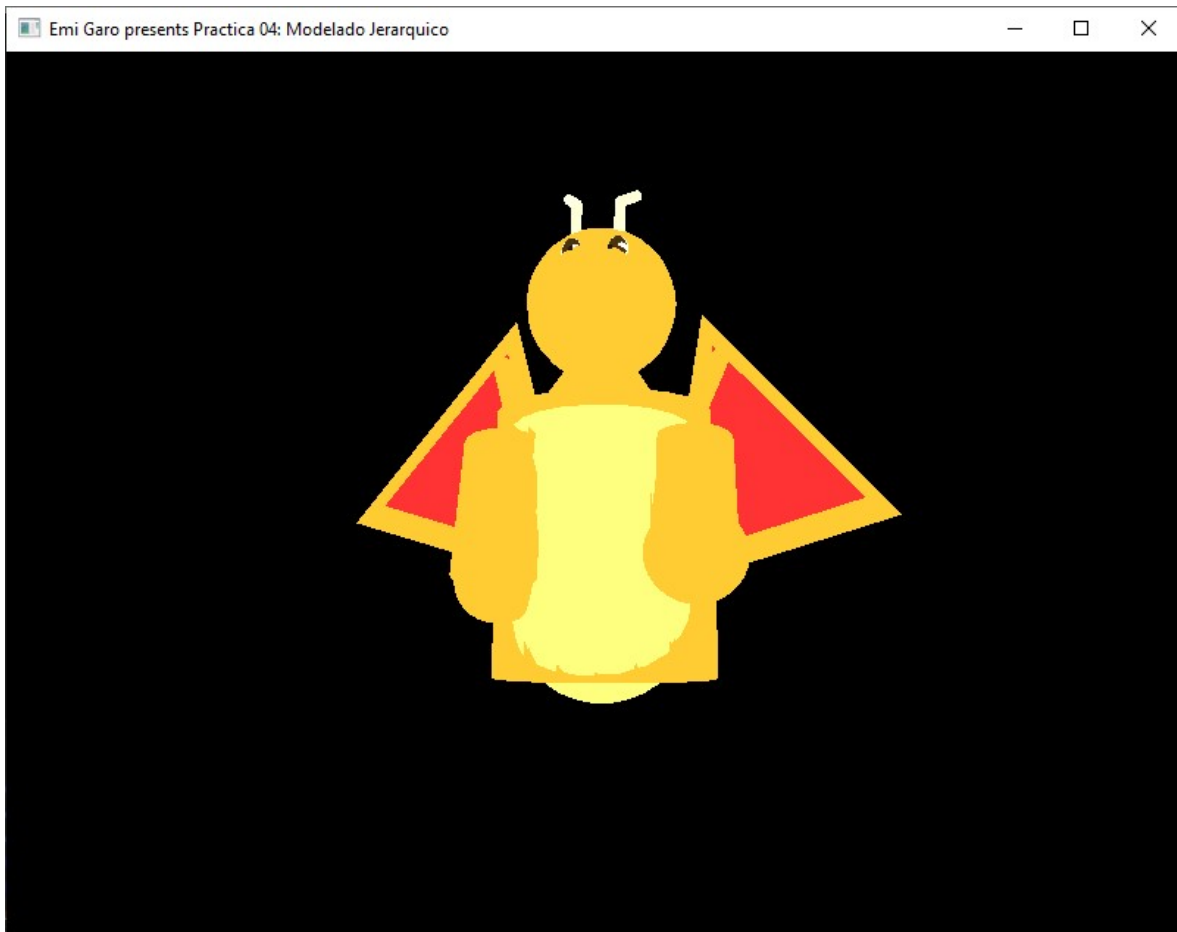
Ahora, modelamos las primeras piezas con jerarquia, que serian los brazos y manos. De nuevo, el mismo principio de la grua y mas facil ya que estas no necesitan moverse, por lo que las manos solo heredan de los brazos.

```
// Left Arm
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.8f, 1.0f, 0.55f));
model = glm::rotate(model, -15 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 1.2f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
// Left Hand
model = glm::translate(model, glm::vec3(-0.0f, -0.5f, -0.25f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
// Right Arm
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.8f, 1.0f, 0.55f));
model = glm::rotate(model, -15 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.35f, 1.2f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry();
// Right Hand
model = glm::translate(model, glm::vec3(0.0f, -0.5f, 0.25f));
model = glm::scale(model, glm::vec3(0.4f, 0.4f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
```

Ahora Dragonite ya tiene brazos, y podemos seguir modelando los pies y cola, los cuales ademas de contener jerarquias pueden moverse. Con

```
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()),
glm::vec3(0.0f, 0.0f, 1.0f));
```

Deberemos de ser extra cuidadosos asi de guardar matrices auxiliares y saber en que etapa de la transformacion estamos para poder modelar las partes, las articulaciones, y la siguiente parte de manera que podamos mover estas y tenga sentido.



El código utilizado para las patas es el siguiente (siguiendo el mismo principio que hemos trabajado):

```
// Right Leg Upper
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-0.8f, 0.0f, 0.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion3()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
// Right Leg
model = modelaux;
model = glm::translate(model, glm::vec3(-0.5f, -0.6f, 0.5f));
model = glm::rotate(model, -8 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 1.2f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
```



```

glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,
2.cilindro, 3.cono, 4.piramide cuadrangular
// Right leg articulation
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.6f, -0.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion4()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//sp.render();
// Right Leg
model = modelaux;
model = glm::translate(model, glm::vec3(-0.0f, -0.1f, -0.2f));
model = glm::rotate(model, 38 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.35f, 0.8f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,
2.cilindro, 3.cono, 4.piramide cuadrangular
// Right leg articulation foot
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.4f, 0.01f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion5()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//sp.render();
// Right foot
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.1f, 0.3f));
model = glm::rotate(model, -31 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh(); // 0.Cubo, 1.piramide triangular, 2.cilindro,
3.cono, 4.piramide cuadrangular

// Left Leg Upper
model = glm::mat4(1.0);

```

```

model = glm::translate(model, glm::vec3(0.8f, 0.0f, 0.2f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion6()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.7f, 0.7f, 0.7f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
sp.render();
// Right Leg
model = modelaux;
model = glm::translate(model, glm::vec3(-0.5f, -0.6f, 0.5f));
model = glm::rotate(model, -8 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.5f, 1.2f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,
2.cilindro, 3.cono, 4.piramide cuadrangular
// Left leg articulation
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.6f, -0.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion7()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//sp.render();
// Left Leg
model = modelaux;
model = glm::translate(model, glm::vec3(-0.0f, -0.1f, -0.2f));
model = glm::rotate(model, 38 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.35f, 0.8f, 0.35f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,
2.cilindro, 3.cono, 4.piramide cuadrangular
// Left leg articulation foot
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, -0.4f, 0.01f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion8()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

```

```

glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//sp.render();
// Left foot
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.1f, 0.3f));
model = glm::rotate(model, -31 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.8f, 0.4f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[0]->RenderMesh(); // 0.Cubo, 1.piramide triangular, 2.cilindro,
3.cono, 4.piramide cuadrangular

```

Y el de la cola el siguiente:

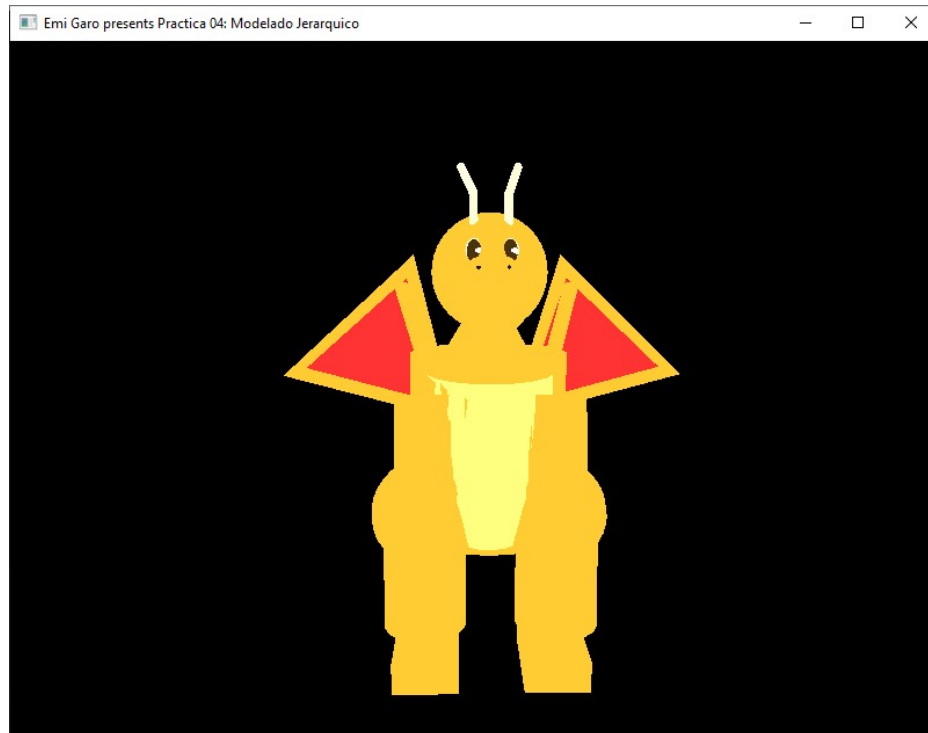
```

// Tail
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 0.3f, -1.3f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::rotate(model, -40 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 1.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[2]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,
2.cilindro, 3.cono, 4.piramide cuadrangular 5.body
// Articulation
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.6f, -0.55f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()),
glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = model;
model = glm::scale(model, glm::vec3(0.2f, 0.2f, 0.2f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//sp.render();
// Tail end
model = modelaux;
model = glm::translate(model, glm::vec3(0.0f, 0.7f, -0.3f));
model = glm::rotate(model, -30 * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.27f, 1.9f, 0.27f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniformMatrix4fv(uniformView, 1, GL_FALSE,
glm::value_ptr(camera.calculateViewMatrix()));
color = glm::vec3(1.0f, 0.8f, 0.2f);

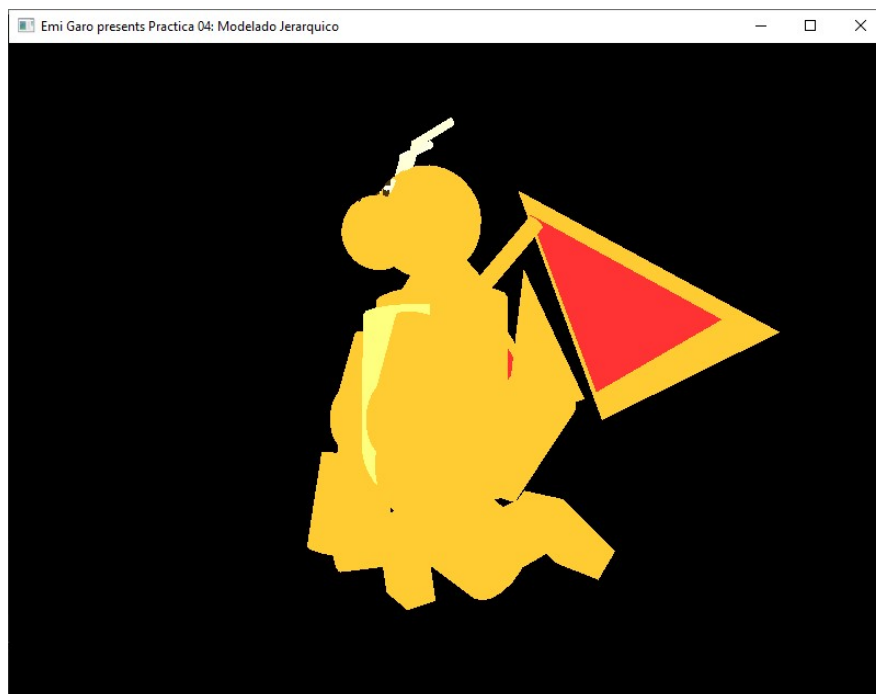
```

```
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
meshList[3]->RenderMeshGeometry(); // 0.Cubo, 1.piramide triangular,  
2.cilindro, 3.cono, 4.piramide cuadrangular 5.body
```

Finalmente compilamos y ejecutamos y tenemos nuestro Dragonite.



Y si presionamos nuestras teclas de control (F, G, H, J, K, L, N, M). Podemos articularlo en diferentes poses.



Conclusión

Durante la practica logramos utilizar los conceptos de modelado jerarquico para que apartir de las primitivas trabajadas anteriormente y las cuales son las mas basicas, pudieramos realizar, no solo contrucciones de modelos mas complejos de representar, si no que podemos ahora manipularlos de diferentes formas, según lo establecimos en la jerarquia que nos permita realizar las transformaciones que necesitemos.

Esto solo fue posible debido al entendimiento que tuvimos de cómo estas se relacionaban entre si utilizando una estructura de padre, hijo donde pudimos abstraerlo en una grua y un modelo de nuestra creacion donde teniamos una extremidad, conectada por una articulacion y conectada a la siguiente extremidad.

Esto no solo permitio rotaciones complejas, si no que debido a tener un nodo padre, pudimos mover este con el nuevo concepto de definir nuestras propias teclas de control para simular movimiento en el espacio tridimensional. Lo cual nos sera util en practicas y proyectos futuros.

Referencias:

- Apuntes de la clase de teoria y laboratorio de “Computacion Grafica e Interaccion Humano Computadora”